

Merge as Graph Extension

Sasson Margalio

June 2026

Abstract

This article proposes a formal interpretation of Merge as graph extension inside the Workspace. I suggest representing the Workspace itself as a graph. Elementary syntactic objects are represented as nodes, while complex syntactic objects formed by Merge are represented as subgraphs of the Workspace, especially as paths between boundary nodes. When Merge applies, it adds a new accessible path to the graph.

The article proposes five internal Layers of the Language Faculty: Semantic Component, A Syntax, A-bar Syntax, Distributed Morphology, and Phonological Component. The same processing cycle is proposed for all five Layers: Merge extends the Workspace, Relinearization constructs a new Output Graph, and Transfer passes that graph as the initial Workspace of the next Layer. The Layers are traversed in one order in generation and in the reverse order in comprehension. The grammars differ by Layer and by direction, but the basic Expand algorithm remains the same.

A Layer is a persistent component with its own ordered grammar of Extension Rules. Expand is the procedure that takes a Workspace, applies the grammar of the Layer, and returns another Workspace. A Rule written in the form $Left \Rightarrow Right$ is applied by finding the Right side in the Workspace and adding the Left side as a new accessible path.

1. Introduction

The proposal developed here is that the Workspace is represented as a graph, and Merge can be formally interpreted as graph extension. When Merge applies, it adds a new accessible path to that graph. The material already present in the Workspace is preserved, and the newly added path can itself participate in later applications of Merge.

In standard linguistic notation, a rule such as

(1) $VP \Rightarrow V NP$

is naturally understood by linguists as a parsing rule: when a $V NP$ configuration is found, it may be recognized as a VP .

An Extension Rule written as $Left \Rightarrow Right$ is applied by finding the $Right$ side in the Workspace and adding the $Left$ side as a new accessible path.

Each Layer owns its own ordered grammar of Extension Rules. The Layer receives a Workspace. The initial state of that Workspace is called the Lexical Array. The Layer applies Merge repeatedly inside the Workspace, and then returns a new graph through Relinearization. Transfer passes that graph as the initial Workspace of the next Layer.

The same Expand algorithm is proposed for five internal Layers of the Language Faculty, traversed in one order in *generation* and in the reverse order in *comprehension*. The grammars differ from Layer to Layer, but the basic processing cycle remains the same.

The article is organized as follows. Section 2 states the proposed five-Layer architecture. Section 3 defines the Workspace and the Lexical Array. Section 4 introduces the graph notation used in the article. Sections 5–7 define Extension Rules and illustrate Merge as graph extension, first in an abstract example and then inside a larger grammar-like Workspace.

Section 8 states the general form of Layer processing. Section 9 describes Relinearization and Transfer. Section 10 explains how the same algorithm applies in both directions of traversal. Section 11 explains why the proposal is a formal interpretation of Merge. Section 12 states what is left open, and Section 13 concludes.

2. The five internal Layers

The proposal assumes five Layers internal to the Language Faculty:

(2)

Semantic Component (Σ),

A Syntax,

A-bar Syntax,

Distributed Morphology (DM),

Phonological Component (Φ)

These five Layers are distinct from the external performance systems. C-I is the intelligent Agent itself, interacting with other cognitive systems. A-P is the biological articulatory-perceptual performance system. The five Layers are the internal linguistic machinery between them.

This article proposes this five-Layer organization as the internal architecture of the Language Faculty. The full pipeline is:

(3)

C-I / Agent → [Agentic Form] →

Semantic Component (Σ) → [Event Structure] →

A Syntax → [NP Structure] →

A-bar Syntax → [Surface Structure] →

Distributed Morphology (DM) → [Morphophonological Representation] →

Phonological Component (Φ) → [Phonetic Form] →

A-P

The bracketed terms name representations transferred between Layers.

In *generation*, the Layers are traversed in this order:

(4) $\Sigma \rightarrow \text{A Syntax} \rightarrow \text{A-bar Syntax} \rightarrow \text{DM} \rightarrow \Phi$

C-I supplies [Agentic Form] to the Semantic Component.

The Semantic Component supplies [Event Structure] to A Syntax.

A Syntax transfers [NP Structure] to A-bar Syntax.

A-bar Syntax transfers [Surface Structure] to Distributed Morphology.

Distributed Morphology transfers [Morphophonological Representation] to the Phonological Component.

The Phonological Component transfers [Phonetic Form] to A-P.

In *comprehension*, the same Layers are traversed in the reverse order:

(5) $\Phi \rightarrow \text{DM} \rightarrow \text{A-bar Syntax} \rightarrow \text{A Syntax} \rightarrow \Sigma$

A-P supplies [Phonetic Form] to the Phonological Component.

The Phonological Component reconstructs [Morphophonological Representation] and transfers it to Distributed Morphology.

Distributed Morphology reconstructs [Surface Structure] and transfers it to A-bar Syntax.

A-bar Syntax reconstructs [NP Structure] and transfers it to A Syntax.

A Syntax reconstructs [Event Structure] and supplies it to the Semantic Component.

The Semantic Component reconstructs [Agentic Form] and supplies it to C-I.

3. The Workspace as a graph

The term Workspace is taken from Chomsky's 2019 UCLA Lectures. Chomsky introduces the Workspace in order to describe how syntactic objects are available during a derivation. The Workspace contains the elementary syntactic objects, or lexical items, together with objects already constructed by MERGE. Chomsky describes it as "the set including the atoms and anything constructed by Merge" (Chomsky 2019).

MERGE then acts on a Workspace. It takes two syntactic objects that are available in the Workspace and forms a new syntactic object from them. The resulting syntactic object is placed in the Workspace and can participate in later applications of MERGE.

MERGE applies repeatedly. At each stage, MERGE uses syntactic objects that are available in the Workspace and builds a new syntactic object. In one way or another, the syntactic objects used in that application become unavailable for independent reuse. They are now parts of the newly constructed syntactic object.

The derivation continues until the relevant exit condition is reached: there is no further available syntactic object with which the remaining object can be merged.

For the present article, two aspects of this picture are especially important. First, syntactic computation is described in terms of a Workspace, not only in terms of a single tree being built step by step. Second, MERGE changes what is available in that Workspace. It constructs new syntactic objects and makes them available for further computation.

I suggest representing the Workspace itself as a graph. At every step, the graph contains a full record of a derivation.

The nodes of the graph represent elementary syntactic objects, Labels, and control points. Paths in the graph represent syntactic objects or sequences of syntactic objects that are available for further computation. When Merge applies, it acts on paths inside the Workspace and adds new paths to the same Workspace.

This is similar to the Workspace picture reviewed above, but it is not identical to it. In the system proposed here, Merge adds another path to the graph.

The Workspace is a graph that can contain several paths between the same boundary nodes. An application of Merge adds a new path between boundary nodes while leaving earlier paths in place.

At this point it is useful to distinguish the Layer from the procedure that operates in a Layer. A Layer is a persistent component of the architecture. It has its own grammar, understood here as an ordered set of Extension Rules. The Layer itself is not the changing Workspace.

The changing operation is Expand. Expand takes one Workspace as its input, applies the grammar of the Layer, and returns another Workspace. In other words, the Layer owns the grammar; Expand applies that grammar to a Workspace. This distinction will matter below, because the same Expand procedure can be used in all five Layers, while the grammar belongs to the individual Layer.

For each Layer, the Workspace has an initial state. I will call this initial state the Lexical Array of that Layer. It is the state of the Workspace before the first application of Merge in that Layer.

The Lexical Array is not only the representation transferred from the previous Layer. It also contains the resources expected by the grammar of the present Layer. In particular, the Label nodes used by the Extension Rules of the present Layer are already present in the Workspace before Merge begins.

Elementary syntactic objects are represented as nodes. Labels are also represented as nodes, but they are not themselves elementary syntactic objects. A token node may point to a Label node, and that outgoing edge gives the token its Label. Control nodes mark positions such as the beginning and end of a path.

Complex syntactic objects formed by Merge are not represented as single nodes. They are represented as subgraphs of the Workspace. In the examples below, the most important subgraphs are paths between boundary nodes, together with the Label edges attached to the token nodes on those paths.

Merge extends the Workspace by adding new material to the graph. In the simplest cases, it adds fresh token nodes and new edges, thereby creating an additional path between two boundary nodes. The earlier path remains in the graph. The Workspace therefore contains the accumulated result of repeated applications of Merge.

The next section introduces the notation used for these graph Workspaces. It will distinguish token nodes, Label nodes, control nodes, path edges, and the compact notation for additional outgoing Label edges.

4. Nodes, labels, and paths

The abstract graph contains token nodes, label nodes, and control nodes. Nodes are written in the form:

(6) (classname:id)

For example:

(7)

(token:1)

(token:2)

(label:A)

(label:B)

(control:begin)

(control:end)

In the examples in this article, token ids are written as running integers:

(8)

(token:1)

(token:2)

(token:3)

A token is labelled by an outgoing edge from the token node to a label node. The following notation:

(9) (token:1) {label:A}

means that (token:1) has an outgoing edge to (label:A).

A visible sequence is represented by token-to-token path edges:

(10) (token:a) → (token:b)

Additional outgoing edges may be written in curly braces immediately after the source node, before the next path arrow. Thus:

(11) (**control:begin**) → (token:e) {label:E} → (token:f) {label:F} → (**control:end**)

abbreviates a graph containing the path edges:

(12)

(control:begin) → (token:e)

(token:e) → (token:f)

(token:f) → (control:end)

and the label edges:

(13)

(token:e) → (label:E)

(token:f) → (label:F)

A token may have more than one additional outgoing label edge. For example:

(14) (token:1) {label:A} {label:B} → ...

means that (token:1) has outgoing edges both to (label:A) and to (label:B).

The edges are formally untyped. The notation {label:A} is only a compact way to show an additional outgoing edge to the label node (label:A).

5. Extension Rules

Rules retain familiar linguistic notation:

(15) Left => Right

The interpretation is:

(16) find Right, add Left

So, a rule such as:

(17) A => B C

is applied by finding a B C path in the Workspace and adding an A path at the same location.

The rule does not *replace* B C with A. Instead, it extends the Workspace so that A becomes available as an additional path. The same interpretation applies in both directions of traversal. There are different ordered sets of Extension Rules for *generation* and *comprehension*. The direction affects Relinearization and Transfer, but not the basic form of Merge as graph extension.

6. A simple Merge example

Assume the initial Workspace contains:

(18) **(control:begin)** → (token:b) {label:B} → (token:c) {label:C} → **(control:end)**

The notation {label:B} means that (token:b) has an additional outgoing edge to the label node (label:B). Similarly, {label:C} means that (token:c) has an additional outgoing edge to the Label node (label:C).

Thus, the path from (control:begin) to (control:end) contains the label sequence B C.

Now suppose the Layer has the Extension Rule:

(19) $A \Rightarrow B C$

The right side of the rule is found:

(20) $B C$

The matching span is:

(21) $(\text{token:b}) \{\text{label:B}\} \rightarrow (\text{token:c}) \{\text{label:C}\}$

Merge then adds a fresh token node with Label A:

(22) **(control:begin)** $\rightarrow (\text{token:3}) \{\text{label:A}\} \rightarrow$ **(control:end)**

After Merge, the Workspace contains both paths:

(23)

(control:begin) $\rightarrow (\text{token:b}) \{\text{label:B}\} \rightarrow (\text{token:c}) \{\text{label:C}\} \rightarrow$ **(control:end)**

(control:begin) $\rightarrow (\text{token:3}) \{\text{label:A}\} \rightarrow$ **(control:end)**

The two paths are alternative paths between the same boundary nodes.

In this example, the boundary nodes are (control:begin) and (control:end). More generally, Merge can apply between any two nodes X and Y , provided there is a path from X to Y . Merge finds a path between two boundary nodes and adds another path between those same boundary nodes.

7. Merge inside a larger Workspace

The previous example used (control:begin) and (control:end) as the boundary nodes. That is only the simplest case. In general, Merge applies anywhere in Workspace.

Consider a simplified but more grammar-like example:

(24) I read a book

Suppose the Workspace contains the following path:

(25)

(control:begin) → (token:I) {label:pronoun} → (token:read) {label:verb} →
(token:a) {label:Det} → (token:book) {label:noun} → (control:end)

and the Layer contains the Extension Rule:

(26) DP => Det noun

For this application of Merge, the boundary nodes are passed as parameters:

(27) X = (**token:read**), Y = (**control:end**)

Between these two boundary nodes X and Y, the Workspace contains the path:

(28) (token:a) {label:Det} → (token:book) {label:noun}

Merge then adds a fresh token node (token:55) with Label DP and adds an additional path between the same boundary nodes:

(29) (**token:read**) → (token:55) {label:DP} → (**control:end**)

After this application of Merge, the Workspace contains both paths (30) and (31):

(30)

(token:read) {label:verb} →
(token:a) {label:Det} →
(token:book) {label:noun} →
(control:end)

(31)

(token:read) {label:verb} →
(token:55) {label:DP} →
(control:end)

Workspace is not limited to the material directly involved in the current application of Merge. It contains all accumulated syntactic objects in the graph. A particular application of Merge receives two boundary nodes as parameters, searches for the right side of a rule between those boundaries and adds the left side as a new path between the same boundaries.

8. Layer grammars and processing

A Layer has its own grammar. In the present proposal, the grammar of a Layer is an ordered set of Extension Rules. The order matters, because Merge operations are applied by scanning the rules of the Layer's grammar.

The same processing procedure is used in every Layer. What changes from Layer to Layer is the grammar: the ordered set of Extension Rules available in that Layer.

Internally, processing proceeds as follows:

(32)

- Receive a Workspace.

- Treat the initial state of that Workspace as the Lexical Array of the Layer.

- Use the ordered grammar of the Layer to apply Merge repeatedly inside the Workspace.

- Relinearize the resulting Workspace.

- Return a new graph as the Output Graph.

The Workspace received by a Layer is not only the material transferred from the previous Layer. It also contains the Label nodes expected by the grammar of the current Layer. These Label nodes are already present when Merge applies. A particular application of Merge may introduce a fresh token node and link it to an existing Label node, but it does not need to create the Label node at that moment.

Thus, a Layer is defined by two things: a Workspace on which it operates, and a grammar that determines which Extension Rules can apply. The processing procedure itself remains the same.

9. Relinearization and Transfer

Relinearization is the construction of a new Output Graph from selected paths in the extended Workspace.

After Merge operations have extended the Workspace, there may be several paths available. Some paths may be interpretable by the next Layer; others may not. Relinearization walks complete paths from (control:begin) to (control:end), extracts the output label sets that are interpretable by the next Layer, and builds a new graph.

The Output Graph is a new Workspace. It is not the same Workspace with some paths marked as visible or invisible. It contains equivalently named nodes, but it is a new graph object constructed by Relinearization.

Transfer is the passing of the Output Graph of one Layer as the initial Workspace of the next Layer.

When the next Layer receives this Workspace, the transferred material supplies the initial state of the next Layer. All the Label nodes expected by the grammar of the next Layer are also present in that Workspace before Merge begins.

Thus, Expand extends the Workspace by repeated applications of Merge, relinearizes the extended Workspace into a new Output Graph, and transfers that Output Graph as the initial Workspace for the next Layer.

10. The same algorithm in both directions

The five Layers can be traversed in two directions.

In generation, the order is:

(33) $\Sigma \rightarrow A \text{ Syntax} \rightarrow A\text{-bar Syntax} \rightarrow DM \rightarrow \Phi$

Generation begins from a Workspace supplied by C-I and proceeds toward A-P.

In comprehension, the order is reversed:

(34) $\Phi \rightarrow \text{DM} \rightarrow \text{A-bar Syntax} \rightarrow \text{A Syntax} \rightarrow \Sigma$

Comprehension begins from a Workspace supplied from the A-P side and proceeds toward C-I. The grammar certainly differs from Layer to Layer, and it also differs by direction. For example, the ordered set of Extension Rules used by a Layer in generation is not expected to be identical to the ordered set of Extension Rules used by that Layer in comprehension.

Nevertheless, the algorithmic form remains the same: Expand, Relinearize, Transfer.

The direction of traversal affects Relinearization and Transfer, but not the basic form of Merge as graph extension. In both directions, a Layer receives a Workspace, applies Merge through the ordered grammar of that Layer, constructs a new Output Graph by Relinearization, and transfers that graph as the initial Workspace for the next Layer.

This is especially important for generation. Generation is not treated here as the brute-force production of all possible lower-level realizations from an abstract object. A specific Workspace has already been supplied by C-I or by the previous Layer. The Layer recognizes structures in that Workspace and restates them in the terms needed for the next Layer.

11. Merge can be understood as graph extension

The proposal is that Merge itself can be formally interpreted as graph extension inside the Workspace.

In the Workspace conception of Merge, syntactic objects already available in the Workspace are used to make a new syntactic object available.

Consider again the grammar-like rule:

(35) $\text{DP} \Rightarrow \text{Det noun}$

When the Right side of this Rule is found between two boundary nodes, the token nodes on that selected path are not removed from the Workspace. Merge adds another path between the same boundary nodes. The added path makes a new syntactic object available in the Workspace.

In the example from the previous section, the selected path was:

(36) (token:a) {label:Det} → (token:book) {label:noun}

and Merge added the path:

(37) (token:read) {label:verb} → (token:55) {label:DP} → (control:end)

The token nodes (token:a) and (token:b) remain in the Workspace. The fresh token (token:55) is added. The new path is an additional accessible path in the Workspace, and it can be used by later applications of Extension Rules during procedure Expand.

After adding a new path, that path may make further Rules applicable. Expand continues as long as further applications of the Layer's Extension Rules remain possible. This gives a formal interpretation of the basic intuition behind Merge. Syntactic objects available in the Workspace are used to add another syntactic object available.

The same graph-extension approach is intended to cover Internal Merge as well. If the same Label appears on both sides of the same Rule, this may identify the same token rather than two distinct tokens pointing to the same Label. This is only a brief indication of the intended solution; the detailed treatment of Internal Merge requires separate discussion.

12. What is left open

The present note defines the core operation: Merge as graph extension inside the Workspace. Several parts of the broader theory require separate treatment.

First, Internal Merge requires a fuller formal account. Section 11 indicated how the same graph-extension format may be extended to cases where an element already present in the Workspace also participates in the added path. The details are left for later work.

Second, Relinearization requires a fuller theory. In this article, Relinearization was characterized as the construction of a new Output Graph from selected paths in the extended Workspace. The direction of traversal between Layers affects the details of Relinearization and Transfer, and those direction-sensitive details must be developed separately.

Third, the five Layers require concrete rule sets. The proposal assumes distinct Layer grammars for the Semantic Component, A Syntax, A-bar Syntax, Distributed Morphology, and the Phonological Component. The present article defines the common form of processing, but the rule systems for the individual Layers belong to later work.

Fourth, the broader empirical motivation for the five-Layer division must be developed in detail. The purpose of the present note is to state the architecture and define the operation that makes the architecture formally usable.

The claim made here is therefore narrow but central: the Workspace can be represented as a graph, and Merge can be interpreted as the extension of that graph by additional accessible paths.

If this interpretation is correct, the central operation of syntax does not require a separate tree-building mechanism. A Workspace can be a graph, and Merge can be the operation that extends that graph. The open questions listed above concern the rule systems, the details of Relinearization, and the treatment of Internal Merge. They do not change the basic form of the proposal: syntactic computation proceeds by extending a Workspace.

13. Conclusion

This article has proposed a formal interpretation of Merge as graph extension inside the Workspace. A Layer receives a Workspace. The Layer applies Merge repeatedly based on its grammar. Each application of Merge finds the Right side of a Rule between two boundary nodes and adds the Left side as a new accessible path between the same boundary nodes.

After repeated applications of Merge, Relinearization constructs a new Output Graph. Transfer then passes that Output Graph as the initial Workspace for the next Layer.

The same processing cycle applies across the five internal Layers: Σ , A Syntax, A-bar Syntax, Distributed Morphology, Φ

References

Chomsky, Noam. 2019. The UCLA Lectures (April 29–May 2, 2019). Transcribed and edited by Robert Freidin. <https://ling.auf.net/lingbuzz/005485>

Halle, Morris, and Alec Marantz. 1993. “Distributed Morphology and the Pieces of Inflection.” In *The View from Building 20: Essays in Honor of Sylvain Bromberger*, edited by Kenneth Hale and Samuel Jay Keyser, 111–176. Cambridge, MA: MIT Press.

Margaliot, Sasson. 2017. Minimalist Semantics and Pre-Linguistic Intelligence. [lingbuzz/003409](https://ling.auf.net/lingbuzz/003409). <https://ling.auf.net/lingbuzz/003409>

